CODE.TALKS 2026 · HAMBURG

Onboarding Your Agent

How to integrate agentic engineering into enterprise workflows

Fabian Meyer

whoami

Staff Agentic Engineer @ Cordes & Graefe KG

- Working in our **Human Agentic Lab**
- Incubator for agentic engineering
- Shaping how teams work with agents - tooling and workflows that make the change easier
- Working on **ONLINE PLUS**, our e-commerce platform

off-work: Pladder (my VTT tool), homelab and thinking about AI doomsday

Cordes & Graefe KG

The holding of the **GC-GRUPPE** - wholesale for plumbing, heating & building services. 100+ wholesalers, 920+ locations across Germany, ~20,000 employees.

ONLINE PLUS

The online shop for trade professionals - and the information platform for planners, architects and specifiers.

We're hiring - open roles, come say hi after the talk

I gave this talk six months ago

What changed? Almost everything.

March 2026

- Opus 4.x as main implementer
- **Strict spec-driven development**
- Detailed plans: everything the agent needs
- Focus: get to a working solution

Today

- Opus 5.5
- **More flexible spec-driven development**
- Far less guidance needed
- **Code-review bottlenecks**
- Focus: make sure all these changes are good

What we kept from spec-driven development

The shape of things.



Ticket

Every Jira issue has the same structure - one that fits *you* and gives the agent done criteria.





Plan

For bigger work, plans that look the same every time. Small stuff? Skip it.





Merge Request

A fixed format reviewers can rely on. More on that later.

Consistency → Comparability

More uniform data to improve your agentic engineering.

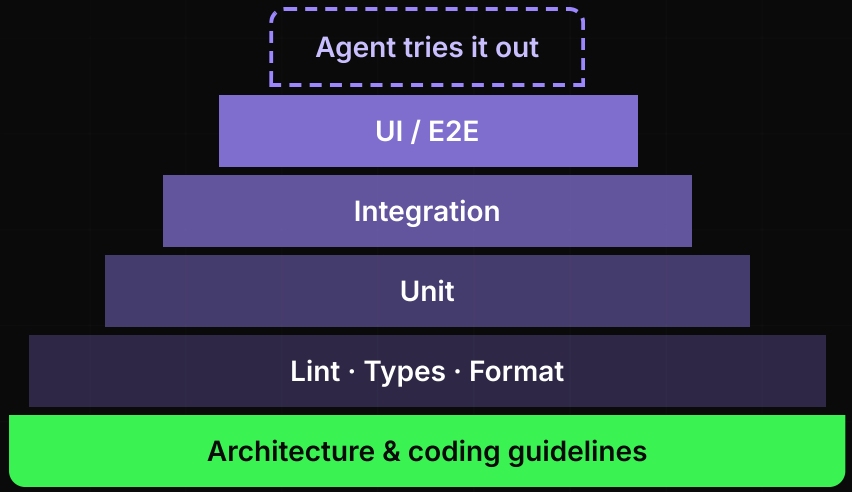
Matt Pocock, Poteto, and many others agree:

Verification is the new game.

Everything you can throw at the agent to make sure the change...

- ...actually **works**
- ...follows **your guidelines**
- ...is **safe & secure**
- ...solves **the right problem**

Make it easy for the agent to verify



Layers of verification - standing on your architecture.

Be explicit. Make it executable.

- Every layer rule → an **architecture test**
- Every coding convention → a **lint rule**
- Every behavior → a **unit test**
- Every boundary → an **integration test**
- Every user flow → a **UI test**

There's a tool to test almost anything. For the rest: let the agent **try it out** - browser, simulator, computer use.

Invest in quality before speed takes off.

If it isn't enforced, it's just a suggestion.

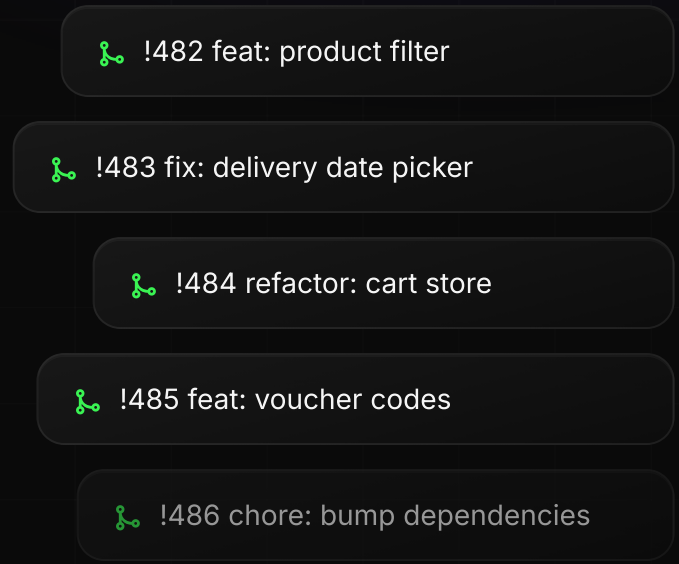
What you leave open, the agent fills in.

- Settle decisions - and **write them down**
- Be **precise** in tickets, plans and prompts
- Say **exactly** what you mean - then you get just that

Clear communication isn't going away. It's becoming more important.

More agents. More code. Code review is the new bottleneck.

- Review is high-effort brain work
- Code you didn’t write
- Without the full context of the repository



Goal: less brain energy per review - without lowering the bar.

Start with a budget

We started from the end: how much code can we review at once?



Lines changed

per merge request - real code, no generated files



Files changed

per merge request



Merge requests

per issue / increment

~~Issue → however many merge requests it takes~~

Merge request budget → how we cut the issue

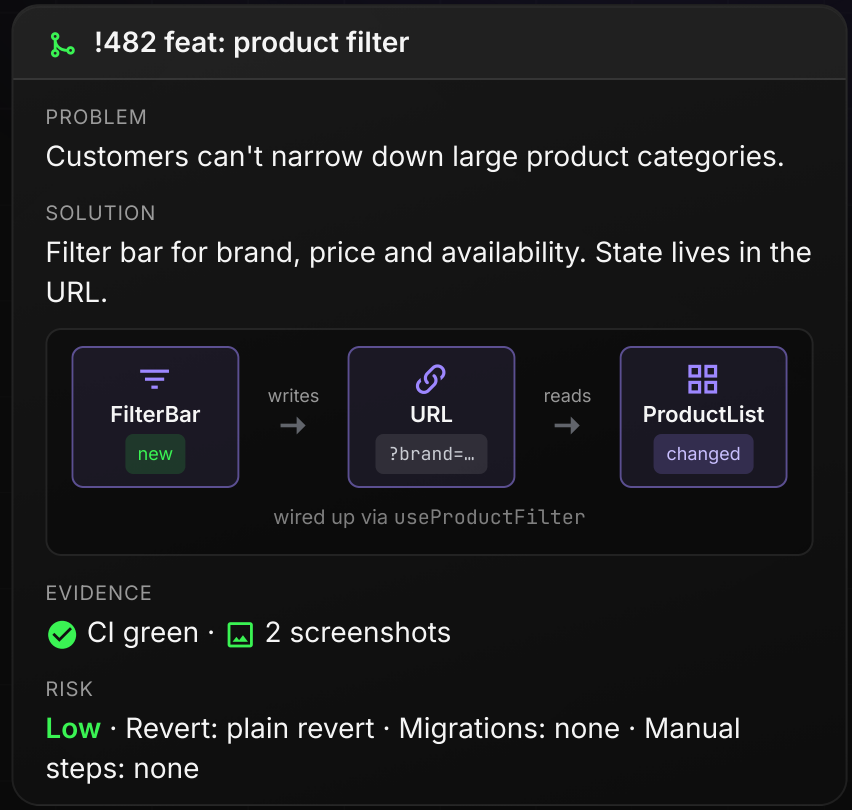
The budget decides the issue size - not the other way round.

A merge request your brain can pre-load

- **Agreed structure** - you know where to look before you look
- **No text walls** - caps on the description
- **Visual first** - the change at a glance
- **Evidence** - prove it works, don't claim it
- **Risk** - how hard is it to revert?

Know how the code works - before you read it.

Then the line-by-line review is far easier - you're checking, not discovering.



WHY REVIEW AT ALL?

~~To find bugs~~

You build understanding of your codebase - without coding in it.

Same sense for how everything fits together. You just build it differently.

Skip it → lose the understanding → **lose control.**

Your new craft: sharpening the agent

Every day, things come up: in review, in the code, in the architecture.

Before

- Review comments, architecture smells, guideline slips
- You used them to sharpen **your own** craft
- Limited to what you saw and remembered

Now

- Same signals - now they sharpen **the agent**
- Into skills, lint rules, tests, guidelines
- The agent can read **every** past merge request

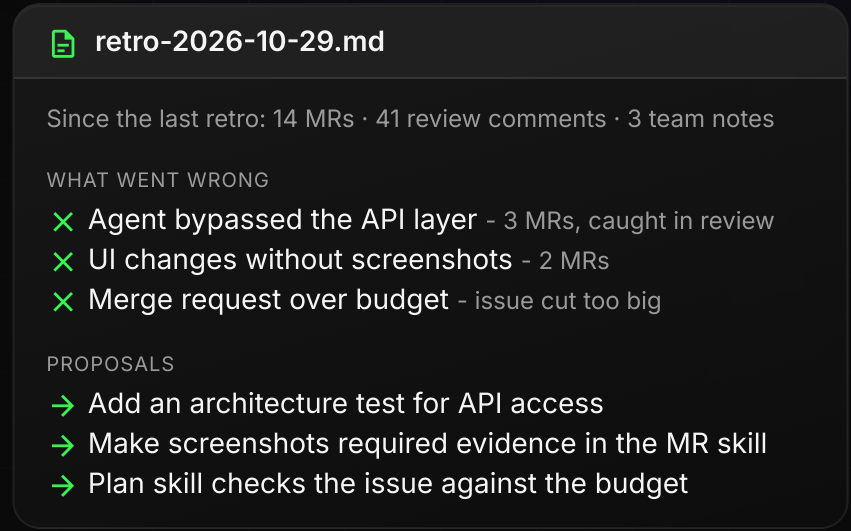
Same information. New ways to collect it. Far more of it.

Learn from every run

One skill per phase of your development lifecycle:

Plan | Implement | Review | **Retro**

- **One person** runs it with the agent
- Collects what went wrong since the last retro: merge requests, reviews
- Plus what each of you noticed
- Prepares a document for the **whole team**
- The team discusses it and changes the workflow



What went wrong becomes a check, a skill, a rule.

You can take this even further for your own work.

every conversation every turn every change

All of it, on your own machine.

An insane amount of data about how you work - for free. Use it.

Congratulations. You lead a small organization now

It lives on your computer. Several agents, several pieces of work - in parallel.



Work in parallel

Agents take time - keep several things going at once. Repeating yourself? Run it on a schedule.



Find the right depth

You can't watch every action - let agents monitor agents. Look closely where it matters.



Sharpen your agents

Like a good lead sharpens their team, you sharpen your agents - and yourself.

Find a balance that works for you.

Where to start

Agree on the shapes

One format for tickets, plans and merge requests. Tickets include done criteria.

Enforce one convention

Pick one thing that went wrong in your last review. Make it verifiable the next time.

Set a merge request budget

Start from how much code you can review at once. Cut issues to fit.

Learn from mistakes

One person runs it with the agent before your next retro. The team changes the workflow.

Every step makes the next agent run easier to verify.

The agent writes the code.

You own the verification and the understanding.

Thank you!

Questions?

Fabian Meyer

STAFF AGENTIC ENGINEER @ CORDES & GRAEFE KG

We're hiring - talk to me if you want to build this with us

LinkedIn

GitHub

CODE.TALKS 2026 · HAMBURG